

Bases de datos

UD10.2. Programación en bases de datos

- Excepciones
- Cursores
- Triggers



1.ÍNDICE

1. GESTIÓN DE ERRORES
 1. GET DIAGNOSTICS
2. EXCEPCIONES
 1. DECLARANDO EXCEPCIONES
 2. DECLARANDO EXCEPCIONES PERSONALIZADAS
 3. LANZANDO EXCEPCIONES
3. CURSORES
4. TRIGGERS

GESTIÓN DE ERRORES

- Podemos gestionar los errores que se producen durante la ejecución de nuestros programas mediante controladores de errores o handlers:

Definición:

```
DECLARE handler_action HANDLER
  FOR condition_value [,
  condition_value] ...
  statement
```

handler_action:

CONTINUE

| EXIT

| UNDO

condition_value:

mysql_error_code

| SQLSTATE [VALUE] sqlstate_value

| condition_name

| SQLWARNING

| NOT FOUND

| SQLEXCEPTION

Acciones:

Las acciones posibles que podemos seleccionar como handler_action son:

CONTINUE: La ejecución del programa continúa.

EXIT: Termina la ejecución del programa.

UNDO: No está soportado en MySQL.

Tipos de errores:

Error Code: Son números específicos de Mysql que no valen para otros gestores.

SQLState Code: Cadena de 5 dígitos basado en ANSI SQL y ODBC y por lo tanto sus códigos son 'estandarizados'.

Códigos de error Mysql:

<https://www.briandunning.com/error-codes/?source=MySQL>

GESTIÓN DE ERRORES

► Uso con **mysql_error_code**:

```
DECLARE CONTINUE HANDLER FOR 1051
BEGIN
    -- instrucciones
END;
```

- Este handler se ejecutará cuando se produzca el error 1051 de MySQL (ocurre cuando se intenta acceder a una tabla que no existe en la base de datos). En este caso, la acción del handler es CONTINUE, por lo que después de ejecutar las instrucciones especificadas en el cuerpo del handler, el procedimiento almacenado continuará su ejecución.

► Uso con la variable **SQLSTATE**:

```
DECLARE CONTINUE HANDLER FOR SQLSTATE '42S02'
BEGIN
    -- instrucciones
END;
```

- cuando se intenta acceder a una tabla que no existe en la base de datos, la variable SQLSTATE toma el valor 42S02. Al igual que en ejemplo anterior, la acción del handler es CONTINUE, por lo que después de ejecutar las instrucciones, el procedimiento almacenado continuará su ejecución.

GESTIÓN DE ERRORES

- ▶ Uso con **SQLWARNING**: (Es equivalente a consultar todos los valores de SQLSTATE que empiezan con 01)

```
DECLARE CONTINUE HANDLER FOR SQLWARNING
BEGIN
    -- instrucciones
END;
```

- ▶ Uso con **NOT FOUND**: (Es equivalente a consultar todos los valores de SQLSTATE que empiezan con 02.)

```
DECLARE CONTINUE HANDLER FOR NOT FOUND
BEGIN
    -- instrucciones
END;
```

- ▶ Se usa, sobre todo, cuando manejamos cursores.

- ▶ Uso con **SQLEXCEPTION**: (Es equivalente a consultar todos los valores de SQLSTATE que empiezan por 00, 01 y 02.)

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
    -- instrucciones
END;
```

GESTIÓN DE ERRORES

► Uso con **SQLSTATE 22012: División por 0**

```
DELIMITER $$
```

```
DROP PROCEDURE IF EXISTS prueba_error_div0c$$
CREATE PROCEDURE prueba_error_div0c(OUT x INT)
BEGIN
    DECLARE division_error INT DEFAULT 0;
    DECLARE resultado DECIMAL(5,2) DEFAULT 0.0;

    DECLARE CONTINUE HANDLER FOR SQLSTATE '22012'
        SET division_error = 1;

    SET x= 0;
    SET resultado = 10 / 0;

    SET x=1;
    IF division_error = 1 THEN
        SELECT "¡Error! División por cero." AS Mensaje;
    ELSE
        SELECT resultado;
    END IF;
```

```
END$$
```

```
DELIMITER ;
```

```
CALL prueba_error_div0c(@xc);
SELECT @xc;
```

```
DELIMITER $$
```

```
DROP PROCEDURE IF EXISTS prueba_error_div0e$$
CREATE PROCEDURE prueba_error_div0e(OUT x INT)
BEGIN
    DECLARE error VARCHAR(255) DEFAULT '';
    DECLARE resultado DECIMAL(5,2) DEFAULT 0.0;

    DECLARE EXIT HANDLER FOR SQLSTATE '22012'
        BEGIN
            SET error = "¡Error! División por cero." ;
            SET x= 2;
            SELECT resultado, error, x;
        END;

    SET x= 0;
    SET resultado = 10 / 0;

    SET x=1;

    SELECT error, x;
```

```
END$$
```

```
DELIMITER ;
```

```
CALL prueba_error_div0e(@xe);
SELECT @xe;
```

GESTIÓN DE ERRORES

- Ejemplo para gestionar el error que se produce al tratar de introducir un registro en una tabla y la clave primaria está duplicada con CONTINUE: (valor de SQLSTATE '23000')

```
CREATE DATABASE pruebahandler;  
USE pruebahandler;  
  
CREATE TABLE t (s1 INT, PRIMARY KEY (s1));  
  
CREATE PROCEDURE gestion23000 ()  
  BEGIN  
    DECLARE x, x2 INT;  
    SET x = 0;  
    SET x2 = 0;  
    DECLARE CONTINUE HANDLER FOR SQLSTATE '23000' SET x2 = 1;  
    SET x = 1;  
    INSERT INTO t VALUES (1);  
    SET x = 2;  
    INSERT INTO t VALUES (1);  
    SET x = 3;  
  END;  
  $$  
  
CALL gestion23000()$$
```

- ¿Qué valores tienen x y x2 al final de la ejecución?

GESTIÓN DE ERRORES

- Vamos a gestionar el mismo error pero con un EXIT: (recordamos que el valor de SQLSTATE '23000')

```
USE pruebahandler;  
Delete from t;  
commit;  
  
CREATE PROCEDURE gestion23000E ()  
BEGIN  
    DECLARE x, x2 INT;  
    SET x = 0;  
    SET x2 = 0;  
    DECLARE EXIT HANDLER FOR SQLSTATE '23000' SET x2 = 1;  
    SET x = 1;  
    INSERT INTO t VALUES (1);  
    SET x = 2;  
    INSERT INTO t VALUES (1);  
    SET x = 3;  
END;  
$$  
  
CALL gestion23000E()$$
```

- ¿Qué valores tienen x y x2 al final de la ejecución?

GESTIÓN DE ERRORES

- Uso con TRANSACCIONES. Podemos utilizar el manejo de errores para decidir si hacemos ROLLBACK de una transacción. En el siguiente ejemplo vamos a capturar los errores que se produzcan de tipo SQLEXCEPTION y SQLWARNING.

```
DELIMITER $$
CREATE PROCEDURE transaccion_en_mysql()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    -- ERROR
    ROLLBACK;
  END;

  DECLARE EXIT HANDLER FOR SQLWARNING
  BEGIN
    -- WARNING
    ROLLBACK;
  END;

  START TRANSACTION;
  -- Sentencias SQL
  COMMIT;
END
$$
```

GET DIAGNOSTICS

- ▶ La sentencia GET DIAGNOSTICS se utiliza para obtener información del entorno de ejecución actual. Principalmente nos facilita información de la sentencia de SQL que se acaba de ejecutar. Es por eso que os resulta muy útil para tratar errores recientes y gestionarlos como excepciones. Se puede usar dentro de una especie de manejador de excepciones genérico. Podemos capturar el mensaje de error en cada momento y userlo en una table de LOG.
- ▶ La sintaxis usada en este caso podría ser:

```
DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
BEGIN
    GET DIAGNOSTICS CONDITION 1
        @code = RETURNED_SQLSTATE, @msg = MESSAGE_TEXT, @errno = MYSQL_ERRNO;
    ROLLBACK;

    INSERT INTO tabla_de_log (campo1, campo2, ....., error_code, error_no, error_text)
    VALUES (valor1, valor2, ....., @code, @errno, @msg);
    COMMIT;
END;
```

Lo que hace básicamente esta sentencia es recoger en variables de sesión los valores diagnósticos del error producido para poder irlos insertando en una table de log sin detener la ejecución (CONTINUE HANDLER).

GET DIAGNOSTICS

- ▶ Ejemplo con SQLSTATE 22012: División por 0

```
CREATE TABLE ej_log(  
  valor_x INT,  
  valor_resultado  
    DECIMAL(5,2),  
  error_code INT,  
  error_no INT,  
  error_text VARCHAR(255));
```

```
DELIMITER $$
```

```
DROP PROCEDURE IF EXISTS prueba_error_div0c2$$  
CREATE PROCEDURE prueba_error_div0c2(OUT x INT)  
BEGIN
```

```
  DECLARE division_error INT DEFAULT 0;  
  DECLARE resultado DECIMAL(5,2) DEFAULT 0.0;
```

```
  DECLARE CONTINUE HANDLER FOR SQLEXCEPTION  
  BEGIN
```

```
    GET DIAGNOSTICS CONDITION 1
```

```
      @code = RETURNED_SQLSTATE, @msg = MESSAGE_TEXT, @errno = MYSQL_ERRNO;
```

```
      select @code, @msg, @errno;
```

```
      INSERT INTO ej_log VALUES (x, resultado, @code, @errno, @msg);
```

```
      SET resultado = -1;
```

```
    END;
```

```
  SET x= 0;
```

```
  SET resultado = 10 / 0;
```

```
  SET x=1;
```

```
  IF division_error = 1 THEN
```

```
    SELECT "¡Error! División por cero." AS Mensaje;
```

```
  ELSE
```

```
    SELECT resultado;
```

```
  END IF;
```

```
END$$
```

```
DELIMITER ;
```

```
CALL prueba_error_div0c2(@xc);
```

```
SELECT @xc;
```

```
SELECT * from ej_log;
```

EXCEPCIONES

Ya hemos visto cómo gestionar o capturar excepciones provocadas durante la ejecución de programas en SQL programable y que ya están definidas por el SGBD.

Pero, al igual que en otros lenguajes de programación, podemos lanzar excepciones personalizadas definidas por los programadores para nuestro sistema.

En cualquiera de los dos casos, cada excepción va a estar asociada a un número ya predefinido por el gestor.

Para hacer mas legible el manejo de excepciones, puedo asociar dicho número a un nombre de excepción y después controlar la captura por dicho nombre y no por el número asociado.

Por ejemplo, en MySQL, el error que aparece cuando se intenta borrar una tabla es el 1051. Dentro de un procedimiento voy a poder capturar esta excepción, pero en vez de capturarla por su número (1051) puedo hacerlo por un nombre, es decir, podemos asignar nombres a los códigos de error.

Podremos hacer esto declarando las excepciones.

DECLARANDO EXCEPCIONES

Siguiendo con el ejemplo anterior, vamos a declarar la excepción para el código de error 1051. Le daremos un nombre, es decir, puesto que podemos asignar nombres a los códigos de error.

Usaremos una sentencia DECLARE :

```
DECLARE <nombre de la excepción> CONDITION FOR <código de error>;
```

Llevada a nuestro ejemplo

```
DECLARE no_existe_tabla CONDITION FOR MYSQL_ERROR_CODE 1051;
```

En el caso de querer dar un nombre a la excepción empleando el SQLState la sintaxis cambia:

```
DECLARE no_existe_tabla CONDITION FOR SQLSTATE '42S02';
```

LANZANDO EXCEPCIONES

Para lanzar desde el código excepciones, usaremos la instrucción SIGNAL.

```
SIGNAL <nombre de la excepción >  
[SET MESSAGE_TEXT = 'Mensaje de error',  
  MYSQL_ERRNO = <Error Code>;]
```

```
SIGNAL SQLSTATE '45000'  
[SET MESSAGE_TEXT = 'Mensaje de error';]
```

Por ejemplo:

```
SIGNAL SQLSTATE '42S02'  
      SET MESSAGE_TEXT = 'Esa tabla no existe',  
      MYSQL_ERRNO = 1051;
```



Si le pongo nombre a SQLSTATE:

```
DECLARE no_existe_tabla CONDITION FOR SQLSTATE '42S02';
```

```
SIGNAL no_existe_tabla  
      SET MESSAGE_TEXT = 'Esa tabla no existe',  
      MYSQL_ERRNO = 1051;
```

DECLARANDO EXCEPCIONES PERSONALIZADAS

Hasta ahora hemos usado excepciones que vienen definidas por el SGBD, pero también podemos generar excepciones definidas por el programador.

Este tipo de excepciones se generan igual que el resto de excepciones, la única precaución que debemos de tener es que el código de error no coincida con los ya existentes.

Existe un código estándar para indicar que la excepción está definida por el usuario (no es ninguna de las que ya existen): **45000**

Por ejemplo, imaginemos que en nuestro programa estamos considerando la altura de un usuario. Se considerará que la altura en cm. de cualquier persona estará comprendida entre 0 y 300. Puedo controlar con una excepción que no se introduzca un valor de la altura incorrecto:

```
DECLARE altura_invalida CONDITION FOR SQLSTATE '45000';
```

Más adelante en el programa, lanzaremos esta excepción si se dan una serie de condiciones. A continuación, veremos cómo se lanzan.

EJEMPLO EXCEPCIONES PERSONALIZADAS

```
DELIMITER $$
```

```
CREATE PROCEDURE verificar_valor(IN numero INT)
BEGIN
    IF numero < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'El número no puede ser negativo';
    END IF;
END $$
```

```
DELIMITER ;
```

```
DELIMITER $$
```

```
CREATE PROCEDURE procesar_valor(IN numero INT)
BEGIN
    -- Llamamos al procedimiento verificar_valor
    CALL verificar_valor(numero);

    -- Si no hay error, mostramos un mensaje
    SELECT CONCAT('El número ', numero, ' es válido y ha sido procesado.') AS mensaje;
END $$
```

```
DELIMITER ;
```

EJEMPLO EXCEPCIONES

```
DELIMITER $$
```

```
CREATE PROCEDURE validar_edad(IN edad INT)
```

```
BEGIN
```

```
-- Definimos una excepción personalizada para edades no válidas
```

```
DECLARE edad_invalida CONDITION FOR SQLSTATE '45000';
```

```
-- Verificamos la edad
```

```
IF edad < 0 OR edad > 120 THEN
```

```
    SIGNAL edad_invalida
```

```
    SET MESSAGE_TEXT = 'Edad no válida: Debe estar entre 0 y 120 años';
```

```
END IF;
```

```
-- Si la edad es válida, mostramos un mensaje
```

```
SELECT CONCAT('Edad ', edad, ' válida.') AS mensaje;
```

```
END $$
```

```
DELIMITER ;
```

CURSORES

Un cursor es un mecanismo de control que permite recuperar filas de una consulta y procesarlas de manera secuencial. En lugar de obtener todo el conjunto de resultados de una consulta en memoria de una sola vez, puedes usar un cursor para obtener una fila a la vez y luego realizar operaciones con ella (como actualizar, insertar, o manipular datos). Se usan dentro de un bloque y habitualmente se recorren con un WHILE (los explícitos).

Propiedades de los cursores en Mysql:

- ▶ Asensitive: el contenido del cursor no es una copia, de modo que una vez abierto, no se puede garantizar que no se vea afectado por modificaciones de los datos realizadas por otros procesos. En mysql, los cursores son asensitive, pero en otros SGBDs podemos encontrar cursores Insensitive. En estos últimos, una vez abierto el cursor, cualquier cambio que se produzca sobre los datos contenidos en el mismo no se visualiza en el resultado. Es como si fuera una copia de sólo lectura de los datos y luego se volcara la información.
- ▶ Read only: son de sólo lectura. No permiten actualizar los datos.
- ▶ Nonscrollable: sólo pueden ser recorridos en una dirección y no podemos saltarnos filas.
- ▶ Cuando declaramos un cursor dentro de un procedimiento almacenado debe aparecer antes de las declaraciones de los manejadores de errores (HANDLER) y después de la declaración de variables locales.

CURSORES implícitos

- ▶ Los cursores **implícitos** son aquellos que no se declaran explícitamente y que nos traen información de la base de datos. Normalmente afectan a un solo registro.
- ▶ Se están usando cursores implícitos cuando realizamos cualquier operación DML sobre la base de datos (DELETE, UPDATE, INSERT).
- ▶ Cuando no necesitamos traer más que un valor totalizado de una consulta o información sobre un solo registro, estamos usando cursores implícitos. Desde cualquier FUNCION o PROCEDIMIENTO, podemos ejecutar una sentencia y almacenar el valor del resultado en una variable. Se modifica ligeramente la sentencia SELECT añadiéndole la cláusula INTO, que indica en qué variables de nuestro programa vamos a almacenar el resultado de la consulta. He aquí dos ejemplos:

```
.....  
DECLARE totalacumulado DECIMAL(15,2);  
  
SELECT SUM(Factura.total)  
  INTO totalacumulado  
  FROM Factura;  
,,,,,,
```

```
.....  
DECLARE bi, iv, tot DECIMAL(15,2);  
  
SELECT base_imponible, iva, total  
  INTO bi, iv, tot  
  FROM Factura  
 WHERE id_facture = '1';  
,,,,,,
```

- ▶ Los cursores implícitos se usan para almacenar más de un registro y tienen que ser definidos como se verá a continuación.

CURSORES explícitos (operaciones)

- ▶ DECLARE. El primer paso que tenemos que hacer para trabajar con cursores es declararlo. La sintaxis para declarar un cursor es:

```
DECLARE nombre_cursor CURSOR FOR consulta_select;
```

- ▶ OPEN. Después tenemos que abrirlo:

```
OPEN nombre_cursor;
```

- ▶ FETCH. Una vez abierto, podemos ir recorriendo el cursor fila por fila con este comando.

```
FETCH [[NEXT] FROM] nombre_cursor INTO variable1, [, variable2] ...variable;
```

- ▶ Cuando se está recorriendo un cursor y no quedan filas por recorrer se lanza el error NOT FOUND, que se corresponde con el valor SQLSTATE '02000'. Por eso cuando estemos trabajando con cursores será necesario declarar un handler para manejar la llegada al final del mismo

```
DECLARE CONTINUE HANDLER FOR NOT FOUND instrucciones
```

- ▶ CLOSE. Cuando se acaba de trabajar con el cursor, éste se cierra.

```
CLOSE nombre_cursor
```

CURSORES explícitos (operaciones)

Así, cuando usamos cursores, necesitamos definir al menos tres líneas en la sección DECLARE de nuestro programa:

```
DECLARE variable de finalización;  
DECLARE nombre_cursor CURSOR FOR consulta_select;  
DECLARE CONTINUE HANDLER FOR NOT FOUND [instrucciones];
```

- La secuencia de uso de un cursor es algo como:

```
DECLARE varid, varc1 INT; /*otras variables */  
  
DECLARE fin INT DEFAULT FALSE; /*variable de finalización*/  
DECLARE cursor1 CURSOR FOR (select id, c1 from tabla); /*id, c1 son INT*/  
/* cuando no hay más datos que leer, la variable fin se pone a TRUE (condición de fin del cursor)*/  
DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin = TRUE;  
  
OPEN cursor1; /*abrimos el cursor llamado cursor1*/  
FETCH cursor1 INTO varid, varc1 ; /* asigna a las variables varid y varc1 los valores de los campos que  
                               extrae la select del registro que está tratando*/  
WHILE (NOT fin) DO /* mientras haya registros en el cursor realizamos las acciones de dentro*/  
    IF (varid%2 = 0) AND (varc1 <> 0) THEN  
        UPDATE tabla SET c1 = varc1*2;  
    END IF;  
    FETCH cursor1 INTO varid, varc1 ; /* Leo el siguiente registro*/  
END WHILE;  
SELECT "Programa terminado"; /* maestro el mensaje por pantalla*/  
CLOSE cursor1; /*abrimos el cursor llamado cursor1*/  
END;
```

TRIGGERS

- ▶ Los triggers (disparadores) en MySQL son procedimientos almacenados que se ejecutan automáticamente en respuesta a ciertos eventos en una tabla, como INSERT, UPDATE o DELETE. Se utilizan para mantener la integridad de los datos, registrar cambios, validar información, entre otros usos.

```
CREATE
  [DEFINER = { usuario | CURRENT_USER }]
  TRIGGER nombre_trigger
  momento evento
  ON table FOR EACH ROW
  [orden_ej_trigger]
  lista_instrucciones

momento: { BEFORE | AFTER }

evento: { INSERT | UPDATE | DELETE }

orden_ej_trigger : { FOLLOWS | PRECEDES } otro_trigger
```

TRIGGERS

- ▶ Dependiendo del tipo de operación que dispare el trigger, tenemos a nuestra disposición dos estructuras que nos permiten acceder a la información que se está manipulando: OLD y NEW.
- ▶ **OLD**: es una estructura con el mismo formato del registro de la tabla que estamos tratando. Se utiliza en los casos de UPDATE y DELETE para comprobar la información del registro que estamos tratando antes del borrado o la actualización del mismo. Para acceder a un campo de este registro, usaremos una expresión del tipo: `OLD.campo`
- ▶ **NEW**: es también una estructura con el mismo formato del registro de la tabla que estamos tratando. Esta estructura se utiliza en los casos de UPDATE e INSERT para comprobar la información del registro que estamos tratando después de la actualización o la inserción del mismo. Para acceder a un campo de este registro, usaremos una expresión del tipo: `NEW.campo`
- ▶ De forma que si quiero comprobar si el valor de un campo antes y después de una actualización continúa siendo el mismo, podría utilizar la siguiente expresión en el cuerpo del trigger:

```
IF NEW.campo <> OLD.campo THEN
    /* realizar acciones para cuando el campo cambia de valor*/
ELSE
    /* realizar acciones para cuando el campo NO cambia de valor */
END IF;
```

TRIGGERS

▶ Ejemplo(cont).

- ▶ Creamos el Trigger 1 asociado a la tabla alumnos con las características:
 - ▶ Se ejecuta antes de una operación de inserción.
 - ▶ Si el nuevo valor de la nota que se quiere insertar es negativo, se guarda como 0.
 - ▶ Si el nuevo valor de la nota que se quiere insertar es mayor que 10, se guarda como 10.

```
DELIMITER $$  
DROP TRIGGER IF EXISTS trigger_nota_before_insert$$  
CREATE TRIGGER trigger_nota_before_insert  
BEFORE INSERT  
ON alumnos FOR EACH ROW  
BEGIN  
    IF NEW.nota < 0 THEN  
        set NEW.nota = 0;  
    ELSEIF NEW.nota > 10 THEN  
        set NEW.nota = 10;  
    END IF;  
END$$
```

TRIGGERS

▶ Ejemplo(cont).

- ▶ Creamos el Trigger 2 asociado a la misma tabla alumnos con las características:
 - ▶ Se ejecuta antes de una operación de actualización.
 - ▶ Si el nuevo valor de la nota que se quiere actualizar es negativo, se guarda como 0.
 - ▶ Si el nuevo valor de la nota que se quiere actualizar es mayor que 10, se guarda como 10.

```
DELIMITER $$  
DROP TRIGGER IF EXISTS trigger_nota_before_update$$  
CREATE TRIGGER trigger_nota_before_update  
BEFORE UPDATE  
ON alumnos FOR EACH ROW  
BEGIN  
    IF NEW.nota < 0 THEN  
        set NEW.nota = 0;  
    ELSEIF NEW.nota > 10 THEN  
        set NEW.nota = 10;  
    END IF;  
END$$
```

TRIGGERS

- Probamos el efecto de estos triggers cuando tratamos de insertar o actualizar los valores problemáticos:

```
-- Probamos las inserciones
DELIMITER ;
INSERT INTO alumnos VALUES (1, 'Alumno', 'Uno', 'Uno', -5);
INSERT INTO alumnos VALUES (2, 'Alumno', 'Dos', 'Dos', 16);
INSERT INTO alumnos VALUES (3, 'Alumno', 'Tres', 'Tres', 6.1);

-- Consultamos los resultados
SELECT * FROM alumnos;

-- Probamos las actualizaciones
UPDATE alumnos SET nota = -2 WHERE id = 3;
UPDATE alumnos SET nota = 18 WHERE id = 3;
UPDATE alumnos SET nota = 9.3 WHERE id = 3;

-- Comprobamos los resultados
SELECT * FROM alumnos;
```

Preguntas

